CS 442 | Mobile Application Development

WEEK 5

Object-Oriented Dart & a First Look at Flutter

LECTURE + PRACTICE — ~75 MIN

Inheritance, mixins, enums, extensions, static & const

FLUTTER TEASER — ~15 MIN

Meet the counter app: widgets are just classes

Based on Flutter in Action, Ch. 2 (Dart) and the opening of Ch. 3 (Breaking into Flutter)

1

TODAY'S SESSION

Today's Session — Roadmap

OOP DART — ~75 MIN

1. **Inheritance, super & override** ~12 min
2. **Abstract classes, interfaces, mixins** ~12 min
3. **Enums, extensions, operators** ~12 min
4. **Static members & const** ~4 min
5. **Hands-on: Shape hierarchy** ~25 min
6. **Common pitfalls** ~5 min

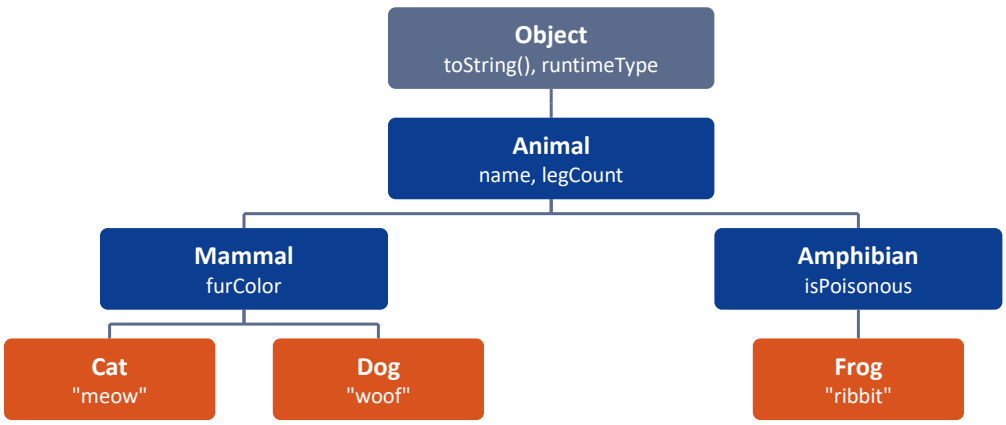
FLUTTER TEASER — ~15 MIN

1. **A widget is just a class** ~5 min
2. **Create & run the counter app** ~5 min
3. **Spot the OOP in main.dart** ~5 min
4. **Self-check & what's next** 2 min

2

INHERITANCE · FIGURE 2.2

Inheritance: A Family Tree of Classes



Read every line as “is a”: a Cat is a Mammal, is an Animal, is an Object

3

INHERITANCE IN CODE

extends: Inherit Everything

Dart

```dart
class Animal {
  String name;
  Animal(this.name);
}
class Dog extends Animal {
  Dog(super.name);
}
var d = Dog('Rex');
print(d.name);         // Rex (inherited)
Animal a = Dog('Max'); // OK: Dog IS-A Animal
Dog x = Animal('Zed'); // ERROR!
```

extends
= take everything from the parent

A Dog IS an Animal
so it fits anywhere an Animal fits

Not the other way
an Animal is not always a Dog

4

CUSTOMISING BEHAVIOUR

super & Overriding Methods

Dart

```dart
class Animal {
  void speak() => print('...');
}
class Dog extends Animal {
  @override
  void speak() {
    super.speak();   // parent's version
    print('Woof!');
  }
}
Dog().speak();  // prints ... then Woof!
```

@override
tells Dart: replace the parent's version

super.speak()
runs the parent's version first

Flutter preview
initState() calls super.initState()

CONTRACTS

Abstract Classes: A Promise With No Body

Dart

```dart
abstract class Shape {
  double area();   // no body: must be provided
}
class Circle implements Shape {
  final double radius;
  Circle(this.radius);
  @override
  double area() => 3.14159 * radius * radius;
}
Shape s = Shape();   // ERROR: abstract
```

abstract
cannot be created directly

It is a contract
area() has no body here

Subclasses deliver
every child must write area()

CONTRACTS · EXAMPLE

Why Abstract? One Contract, Many Forms

Dart

```
abstract class PaymentMethod {
  void pay(double x);   // the contract
}
class Card extends PaymentMethod {
  @override
  void pay(double x) => print('Card: $x');
}
class Cash extends PaymentMethod {
  @override
  void pay(double x) => print('Cash: $x');
}
void checkout(PaymentMethod m) => m.pay(50);
// main: checkout(Card());  checkout(Cash());
```

Contract
PaymentMethod only promises pay()

Many forms
each class writes its own pay()

One caller
checkout() is a plain function, outside any class

7

EXTENDS vs IMPLEMENTS

Two Ways to Build on a Class

extends

- Inherits real, working code
- Fields and method bodies come free
- Only ONE parent allowed
- Use when: There IS a relationship
  - Like a cat is a mammal

implements

- Inherits only the API shape
- You write every method yourself
- Many allowed: implements A, B
- Use when: you promise to fit a contract

Every Dart class is also an interface, so any class can be implemented

8

MIXINS

Mixins: Sharing Code Sideways

Dart

```
class Animal {}
mixin Flyer {
  void fly() => print('Flying!');
}
class Bird extends Animal with Flyer {}
class Plane with Flyer {}   // no Animal!

Bird().fly();   // Flying!
Plane().fly();  // Flying!
```

with
adds a mixin's methods to a class

One parent, many mixins
extends once, with many times

Used all over Flutter
you will meet it very soon

9

CHEAT SHEET

extends · with · implements

`extends`

IS-A relationship.
One parent only.
Inherits real code.

Dog extends Animal

`with`

CAN-DO behaviour.
Many allowed.
Adds ready-made code.

Bird with Flyer

`implements`

PROMISES an API.
Many allowed.
No code inherited.

Circle implements Shape

10

ENUMS

Enums: Basic & Enhanced

Dart

```dart
enum Status { pending, active, done }

enum Planet {
  mercury(3.7), earth(9.8);   // with data
  final double gravity;
  const Planet(this.gravity);
}

print(Planet.earth.gravity);   // 9.8
```

Basic enum
a fixed set of named choices

Safer than strings
'pednig' typo can't compile

Enhanced (Dart 2.17+)
values can carry data

MODERN DART

Extension Methods

Dart

```dart
extension NumberParsing on String {
  int toIntOrZero() =>
      int.tryParse(this) ?? 0;
}

'42'.toIntOrZero();     // 42
'oops'.toIntOrZero();   // 0
```

Add methods to any type
even ones you don't own

No subclass needed
no wrapper object either

Great for small helpers
String, List, DateTime...

OPERATOR OVERLOADING

Teach + to Your Own Class

Dart

```dart
class Vector {
  final double x, y;
  Vector(this.x, this.y);
  Vector operator +(Vector o) =>
      Vector(x + o.x, y + o.y);
}

var v = Vector(1, 2) + Vector(3, 4);
// v is Vector(4, 6)
```

You define what + means
for your own class

Also common
==, [], <, >

Use sparingly
only when the meaning is obvious

STATIC & IMMUTABLE

static Members & const Constructors

Dart

```dart
class MathUtils {
  static const double pi = 3.14159;
  static int square(int n) => n * n;
}
MathUtils.square(5);   // 25 (no object needed)

class Point {
  final int x, y;
  const Point(this.x, this.y);
}
const a = Point(0, 0);   // built at compile time
```

static
belongs to the class, not an object

const constructor
fully constant, built at compile time

Flutter loves const
const MyWidget() is faster

HANDS-ON · 25 MIN

Practice: Build a Shape Hierarchy

1. **Abstract Shape**
   area() with no body, plus describe()
2. **Circle & Rectangle**
   extend Shape; use @override
3. **Square**
   extends Rectangle; super(side, side)
4. **Sortable mixin**
   sort a List<Shape> by area
5. **Bonus**
   const Color field; factory UnitCircle
6. **Share**
   Pairs show their hierarchy (2 min each)

15

HANDS-ON · STARTER CODE

Starter: Fill in the TODOs

Dart

```dart
abstract class Shape {
  double area();
  String describe() => 'Area: ${area()}';
}
class Circle extends Shape {
  final double r;
  Circle(this.r);
  @override
  double area() => 3.14159 * r * r;
}
// TODO: Rectangle, Square, mixin Sortable
```

Paste into DartPad
dartpad.dev

Test it
put 3 shapes in a List<Shape>

Hint for sorting
list.sort((a, b) =>
a.area().compareTo(b.area()))

16

GOTCHAS

Common Pitfalls in OOP Dart

1. **Forgetting @override**
   A typo silently makes a new method

2. **extends vs implements**
   Wanting code? Use extends

3. **Deep inheritance chains**
   Prefer composition: small parts

4. **== without hashCode**
   Breaks Set and Map lookups

5. **Clashing mixin names**
   Two mixins, same method name

6. **Square extends Rectangle?**
   Is-a can break real-world rules